

Keywords: grokking • learning theory • ridge regression • generalization dynamics

To Grok Grokking

The First Mathematical Proof That Delayed Generalization Is an Inevitable Feature of Over-Parameterized Regularized Learning

By Mingyue Xu, Gal Vardi & Itay Safran (Hebrew University of Jerusalem; Ben-Gurion University of the Negev)

arXiv:2601.19791 • ICML 2026 Honorable Mention

Grokking—the strange phenomenon where a neural network first memorizes its training data and then, thousands of steps later, suddenly generalizes —has captivated the machine learning community since its discovery. Yet the phenomenon has resisted rigorous explanation. This paper delivers one: in a classical ridge regression setting, gradient descent with weight decay *provably* must traverse three phases—rapid overfitting, prolonged poor generalization, and eventual near-perfect test accuracy—with explicit quantitative bounds on how long each phase lasts as a function of every hyperparameter.

AT A GLANCE

Problem: Grokking has no rigorous theoretical explanation; practitioners cannot predict when or whether it will occur.

Why it matters: Grokking affects training schedules and regularization choices across many deep learning applications; without a theory, practitioners rely on empirical trial-and-error.

Method: Exact analysis of gradient flow with ℓ_2 regularization on over-parameterized Gaussian linear regression via spectral decomposition of the sample covariance.

Result: End-to-end proof of all three grokking phases; grokking time $T_{\text{grok}} \asymp \lambda^{-1}$ scales inversely with weight decay.

Evidence: Theory-predicted and measured grokking times match within 5% across all tested hyperparameter configurations.

The Big Picture

Grokking was first documented empirically in 2022: small transformers trained on modular arithmetic tasks would rapidly reach 100% training accuracy yet hover near chance-level test accuracy for tens of thousands of additional gradient steps, before abruptly transitioning to near-perfect generalization. The phenomenon was striking because it violated the common understanding that

a model which overfits is simply “stuck”—that continued training without additional data would only worsen generalization.

Subsequent work documented grokking in increasingly diverse settings, from formal grammar to image classification to weight-shared networks. Competing explanations were offered—“efficient circuits” displacing “memorizing circuits,” progressive Fourier feature acquisition, norm minimization dynamics—but all remained heuristic accounts tied to the neural network architecture rather than to the statistical estimation problem underlying them.

This paper strips away the architecture and asks: does grokking occur in the simplest possible supervised learning setting? The answer is yes.

Why Existing Approaches Fall Short

Prior theoretical analyses of grokking were confined to toy settings (e.g., a single neuron or a two-layer network on binary classification) and could not derive exact grokking times. Most empirical studies on grokking used transformers on modular arithmetic, where the discrete structure of the task confounds the statistical learning dynamics.

The “efficient circuit” account holds that grokking reflects the displacement of high-norm memorizing solutions by low-norm generalizing solutions, driven by weight decay. While this story is directionally correct, it does not predict when the displacement occurs or how hyperparameter choices modulate the transition time. The literature on double descent and overfitting-to-generalization transitions in ridge regression provides important background, but none of these works study the *temporal dynamics* of the transition under gradient flow—they analyze stationary solutions only.

Core Mechanism

The mechanism of grokking in ridge regression is the spectral separation between “spurious” and “signal” components of the weight vector.

Early in training, gradient descent minimizes training loss by growing weight components along *all* singular directions of the data matrix, including those with small singular values that correspond to noise. These high-magnitude spurious components allow the model to fit the training data perfectly but generalize poorly.

Weight decay λ then acts as an exponential decay on these components at rate proportional to λ . The signal components (those aligned with large singular values) are also decayed, but they are simultaneously reinforced by the error gradient from training examples that the spurious components cannot fit well. Eventually, as the spurious components shrink below the noise floor, the signal components dominate and generalization improves

rapidly.

Technical Formulation

Let $X \in \mathbb{R}^{n \times d}$ (with $d \gg n$) be the data matrix and $y \in \mathbb{R}^n$ the labels. Gradient flow on the regularized loss $\mathcal{L}(\theta) = \frac{1}{2n} \|X\theta - y\|^2 + \frac{\lambda}{2} \|\theta\|^2$ gives:

$$\dot{\theta}(t) = -\frac{1}{n} X^\top (X\theta(t) - y) - \lambda\theta(t)$$

Decomposing via the SVD $X = U\Sigma V^\top$, with $\bar{y}_i = (U^\top y)_i / \sqrt{n}$ and singular values σ_i , the solution is:

$$\theta(t) = V \operatorname{diag} \left(\frac{\sigma_i^2/n}{\sigma_i^2/n + \lambda} \left(1 - e^{-(\sigma_i^2/n + \lambda)t} \right) \frac{\bar{y}_i}{\sigma_i/\sqrt{n}} \right) \mathbf{1}$$

For small singular values $\sigma_i \rightarrow 0$ (the spurious directions), the weight decays at rate λ once it has reached its early maximum. The grokking time is dominated by this decay:

$$T_{\text{grok}} \asymp \frac{1}{\lambda} \log \frac{\|\theta_{\text{spurious}}^*\|}{\varepsilon}$$

where $\theta_{\text{spurious}}^*$ is the spurious-direction norm at peak overfitting and ε is the target generalization error.

Learning or Inference Procedure

1. Initialize $\theta(0) = 0$.
2. Run gradient flow on $\mathcal{L}(\theta)$ with weight decay $\lambda > 0$.
3. **Phase I (steps 0 to T_{fit}):** Training loss reaches near-zero; test error remains high as spurious components grow.
4. **Phase II (steps T_{fit} to T_{grok}):** Training loss stays near-zero; weight decay erodes spurious components; test error remains high.
5. **Phase III (steps $> T_{\text{grok}}$):** Spurious components drop below noise threshold; signal components dominate; test error drops rapidly to near zero.

The three phases are mathematically sharp: the paper proves that $T_{\text{grok}}/T_{\text{fit}} \rightarrow \infty$ as $\lambda \rightarrow 0$.

What the Guarantee Says

The paper proves the following for over-parameterized ridge regression with $d/n \rightarrow \infty$ and a well-separated signal-to-noise ratio:

- (1) The model *necessarily* overfits: $\mathcal{L}^{\text{train}}(\theta(T_{\text{fit}})) = 0$ with high probability.
- (2) Overfitting *necessarily* persists: there exists $c > 0$ such that $\mathcal{L}^{\text{test}}(\theta(t)) \geq c$ for all $t \in [T_{\text{fit}}, T_{\text{grok}}]$.
- (3) Generalization *eventually* occurs: $\mathcal{L}^{\text{test}}(\theta(t)) \rightarrow 0$ as $t \rightarrow \infty$.

The grokking time $T_{\text{grok}} = \Theta(\lambda^{-1})$ is tight: both the upper and lower bounds are of this order.

Experimental Findings

λ	n	Measured T_{grok}	Predicted
10^{-3}	100	42,800	41,500
10^{-2}	100	4,600	4,300
10^{-1}	100	520	490
10^{-3}	500	39,200	38,000

Theory and experiment agree within 5% across all configurations. The paper also shows that grokking can be *eliminated* entirely by choosing $\lambda \geq \lambda^*$ (derived analytically) or *accelerated* by curriculum learning, where the training order of examples is arranged to minimize the norm of the spurious component.

Ablations and Interpretation

Effect of λ : As λ increases 10-fold, grokking time decreases by approximately 10-fold, confirming the $\Theta(\lambda^{-1})$ prediction. Above a critical $\lambda^* = \sigma_{\min}^2/n$ (smallest signal singular value squared), grokking is eliminated: the model never overfits.

Effect of n : Increasing n (more training data) reduces the norm of the spurious component at overfitting, shortening T_{grok} sublinearly. The theory predicts $T_{\text{grok}} = \Theta(\lambda^{-1} \log n)$ and experiments confirm the logarithmic dependence.

Nonlinear networks: Empirical tests on two-layer ReLU networks trained on the same linear regression task exhibit grokking with qualitatively identical phase structure and λ^{-1} dependence, suggesting the linear analysis captures essential dynamics.

Curriculum learning: Ordering examples by signal-to-noise ratio (easiest first) reduces T_{grok} by 3–4 $\times$ without changing the final test error, consistent with the theory’s prediction that curriculum learning reduces the spurious component norm at overfitting.

Reference

Mingyue Xu, Gal Vardi, Itay Safran. **To Grok Grokking: Provable Grokking in Ridge Regression.** arXiv:2601.19791, January 2026. ICML 2026 Outstanding Paper Honorable Mention. <https://arxiv.org/abs/2601.19791>

Keywords: diffusion models • random matrix theory • generative models • statistical learning

The Seed Knows the Model

Random Matrix Theory Reveals Why Diffusion Models Trained on Different Data Produce the Same Images

By Binxu Wang, Jacob A. Zavatone-Veth & Cengiz Pehlevan (Harvard Kempner Institute; Harvard Society of Fellows; Harvard SEAS)

arXiv:2602.02908 • ICML 2026 Oral / Honorable Mention

Train two diffusion models on non-overlapping halves of ImageNet. Give both the same noise seed. Despite never sharing a single training image, the models produce strikingly similar outputs. This reproducible observation has prompted concern about memorization and dataset leakage—but the true cause is far more mundane and, paradoxically, more fundamental: shared Gaussian statistics across any large natural image dataset already constrain most of what a finite-data denoiser can generate. A random matrix theory framework makes this precise, predicting cross-split consistency from first principles.

AT A GLANCE

Problem: Diffusion models trained on disjoint data subsets generate nearly identical images from the same noise seed, raising memorization and reproducibility concerns.

Why it matters: Understanding the source of this consistency is critical for auditing generative models for data privacy, copyright, and reproducibility.

Method: Random matrix theory analysis of the linear diffusion model; exact formulas for the expected denoiser and its cross-split variance.

Result: Consistency traces to shared Gaussian statistics across splits; the effective noise level is renormalized as $\sigma^2 \rightarrow \kappa(\sigma^2)$ via a self-consistent RMT equation.

Evidence: RMT predictions match empirical cross-split cosine similarity within 2–4% across all tested dataset sizes and aspect ratios.

The Big Picture

Diffusion models generate images by iteratively denoising a noise vector $x_T \sim \mathcal{N}(0, I)$. Because the entire generation process is deterministic given the initial seed x_T , two models starting from the same seed produce the same image if and only if they have learned the same score function. The observation that models trained on *different* data nonetheless produce similar images therefore implies

that the learned score functions agree—which in turn implies that the differences in training data have largely not been captured.

Prior work attributed this phenomenon to memorization (the models share training examples through implicit data augmentation) or to architectural inductive biases (convolutional symmetries, attention patterns). This paper shows neither explanation is necessary: the consistency is predicted by the population-level Gaussian statistics of the data, which are approximately identical across any two large-enough random subsets of natural images.

Why Existing Approaches Fall Short

Memorization-based accounts predict that cross-split consistency should decrease as the two data splits become more diverse. However, the consistency persists even when the splits are drawn from entirely different semantic categories of ImageNet, falsifying this explanation.

Architecture-based accounts predict that consistency should vanish for architectures trained without weight sharing (e.g., plain MLPs). Empirically, consistency is observed across all tested architectures, inconsistent with this account.

Statistical privacy audits (membership inference, differential privacy bounds) focus on individual example memorization rather than population-level structural similarities. None of these frameworks could predict the direction or magnitude of cross-split agreement.

Core Mechanism

The consistency mechanism operates through the spectral structure of the data covariance. In the linear diffusion model, the optimal denoiser at noise level σ is the Wiener filter:

$$s_\sigma(x_\sigma) = -(\Sigma + \sigma^2 I)^{-1} x_\sigma$$

where Σ is the data covariance. Two models trained on different splits both estimate Σ from their respective finite datasets. Their estimates $\hat{\Sigma}^{(1)}$ and $\hat{\Sigma}^{(2)}$ are noisy, but their *mean* over random splits converges to the *same* population covariance Σ . The shared population covariance is the source of cross-split consistency: the two denoisers agree in expectation and differ only in their finite-sample fluctuations, which vanish at rate $O(1/\sqrt{N})$.

Technical Formulation

In the proportional limit $N, d \rightarrow \infty$ with $\gamma = d/N$ fixed, the sample covariance resolvent $(\hat{\Sigma} + \sigma^2 I)^{-1}$ converges to a deterministic function of the population covariance Σ via the self-consistent equation:

$$\kappa(\sigma^2) = \sigma^2 + \gamma \int \frac{\lambda}{\lambda + \kappa(\sigma^2)} d\mu(\lambda)$$

where μ is the spectral measure of Σ . This $\kappa(\sigma^2) > \sigma^2$ is the *effective noise level*: finite-data denoisers experience more noise than they receive, explaining the systematic bias toward the dataset mean.

The expected cross-split agreement of generated samples is:

$$\mathbb{E}[\langle x^{(1)}, x^{(2)} \rangle] = \text{tr}[(\Sigma + \kappa I)^{-2} \Sigma^2] / d$$

Cross-split variance decomposes into three additive terms:

$$\text{Var}[\hat{s}_\sigma(x)] = V_{\text{aniso}} + V_{\text{inhom}} + V_{\text{size}}$$

where V_{aniso} captures eigenmode anisotropy, V_{inhom} captures per-input inhomogeneity, and $V_{\text{size}} = O(1/N)$ is the global $1/N$ scaling.

Learning or Inference Procedure

1. Draw two independent data splits of size N each from $p(x)$.
2. Train a linear diffusion model (Wiener denoiser) on each split, obtaining score functions $\hat{s}_\sigma^{(1)}$ and $\hat{s}_\sigma^{(2)}$.
3. Generate images $x^{(1)}$ and $x^{(2)}$ from the same noise seed x_T using Euler–Maruyama sampling.
4. Measure cross-split cosine similarity $\langle x^{(1)}, x^{(2)} \rangle / (\|x^{(1)}\| \|x^{(2)}\|)$.
5. Compare to the RMT prediction derived from the population covariance Σ and $\kappa(\sigma^2)$.

The RMT prediction requires only Σ and $\gamma = d/N$ — neither the specific split realizations nor any model training is needed.

What the Guarantee Says

The main theorem establishes: as $N, d \rightarrow \infty$ with $\gamma = d/N$ and signal-to-noise ratio bounded away from 0 and ∞ :

- (1) *Expectation*: $\mathbb{E}[\hat{s}_\sigma(x)]$ converges to $-(\Sigma_{\text{eff}} + \kappa I)^{-1}x$ where Σ_{eff} is a deterministic function of the population covariance.
- (2) *Fluctuations*: $\text{Var}[\hat{s}_\sigma(x)] = O(1/N)$, so cross-split agreement is $1 - O(1/N)$ for fixed d/N .
- (3) *Bias*: Generated samples are systematically pulled toward the dataset mean along low-variance principal components, with magnitude $O(\gamma)$ for fixed spectral gap.

Experimental Findings

Setting	Measured	RMT pred.
Linear, $N = 500$	0.47	0.46
Linear, $N = 2000$	0.23	0.22
$\gamma = 2.0$	0.61	0.60
ImageNet PCA approx.	0.38	0.36

All values are cross-split cosine similarity. RMT predictions match observations within 2–4% across all settings.

The framework also correctly predicts *directional* bias: generated samples shift toward the dataset mean along the eigenvectors with smallest eigenvalues, which the Wiener filter systematically undershoots.

Ablations and Interpretation

Dataset size: Cross-split consistency decreases as $O(1/\sqrt{N})$, confirming the $1/N$ fluctuation bound. Practitioners can use this formula to determine the minimum dataset size needed to keep consistency below an acceptable threshold.

Aspect ratio γ : Higher γ (more parameters per sample) increases cross-split consistency. Modern large-latent-space models effectively operate at high γ , explaining why they show stronger consistency than smaller models.

Nonlinear diffusion: Experiments on DDPM with U-Net score models show qualitatively consistent results: cross-split similarity decreases with N and increases with γ , suggesting the linear theory captures the dominant statistical effect even in nonlinear models.

Semantic consistency: Cross-split consistency in semantic embedding space (CLIP) is 2–3 $\times$ larger than in pixel space, suggesting the linear mechanism dominates more strongly at the semantic level where principal components carry more variance.

Reference

Binxu Wang, Jacob A. Zavatone-Veth, Cengiz Pehlevan. **A Random Matrix Theory Perspective on the Consistency of Diffusion Models**. arXiv:2602.02908, February 2026. ICML 2026 Oral / Outstanding Paper Honorable Mention. <https://arxiv.org/abs/2602.02908>

Keywords: reinforcement learning • vision-language models
• reward design • embodied agents

VLM-AR3L

Dual VLM Reward Signals—Absolute and Relative—Unlock Open-World Reinforcement Learning Without Reward Engineering

By Kuan-Chen Chen, Winston Chen, Wei-Fang Sun & Min-Chun Hu (National Tsing Hua University; NVIDIA NVAITC)

arXiv:2607.00483 • IJCAI 2026

Designing reward functions for open-ended reinforcement learning tasks is the field’s most persistent bottleneck. Vision-Language Models offer a promising oracle—they understand goals expressed in natural language and can evaluate visual observations—but single-signal VLM reward methods suffer from either sparse absolute evaluations or noisy pairwise comparisons. VLM-AR3L combines both: an absolute reward model scoring current state quality and a relative reward model measuring inter-frame progress. On four long-horizon Minecraft tasks from MineDojo, the dual-signal framework achieves 29–41% relative improvement over the strongest single-signal VLM baseline.

AT A GLANCE

Problem: Absolute VLM rewards are sparse in long-horizon tasks; relative VLM rewards lack goal-distance information. Neither alone trains effective agents.

Why it matters: Open-ended embodied RL requires dense, well-shaped reward signals without human-designed reward functions.

Method: Separate absolute (scalar state evaluation) and relative (pairwise progress) reward MLPs trained on offline VLM preference labels; combined as weighted sum for PPO policy training.

Result: 29–41% relative improvement over best single-signal VLM baseline on all four MineDojo Minecraft task categories.

Evidence: Ablation confirms both reward branches contribute; removing either degrades success rate by 20–30%.

The Big Picture

Reinforcement learning in open-ended, visually complex environments requires reward signals that are dense enough to provide learning signal throughout each episode and semantically grounded enough to evaluate progress toward linguistically-expressed goals. Human reward engineering meets both requirements but is expensive and

brittle—reward functions designed for one task rarely transfer.

Vision-Language Models have transformed multimodal understanding: they can interpret visual scenes in the context of language instructions, making them natural candidates for reward oracles. However, VLMs are expensive to query at every RL training step (millions of environment interactions). VLM-AR3L addresses this by distilling VLM knowledge into lightweight reward models queried once, offline, before training begins.

Why Existing Approaches Fall Short

EUREKA and similar methods generate only *absolute* scalar rewards: the VLM evaluates each frame independently and assigns a score. In long-horizon tasks (sequences of hundreds to thousands of steps), this signal is near-zero for most of the trajectory—the agent receives almost no feedback until it has nearly achieved the goal. Sparse rewards cause RL algorithms to explore randomly for most of training.

Preference-based methods (RLHF-style relative rewards) compare pairs of states and infer binary preferences. These avoid sparsity but introduce a different pathology: the agent can score well on pairwise comparisons by finding states that look locally better than their neighbors, without making global progress. This leads to oscillatory behavior around local optima.

Prior work that combined absolute and relative rewards did so in domains with dense symbolic state information (chess, video games with score counters). VLM-AR3L is the first to derive both signals from visual observations with natural-language goals using a single VLM.

Core Mechanism

The framework decomposes the reward signal into two complementary streams:

Absolute Reward Model (ARM): A learned scorer $f_{\phi}^{abs} : \mathcal{O} \times \mathcal{G} \rightarrow [0, 1]$ maps a current visual observation x_t and language goal g to a scalar task-achievement estimate. The ARM is stable: even a noisy estimate of absolute task completion prevents catastrophic reward collapse and anchors the agent’s sense of total progress.

Relative Reward Model (RRM): A learned comparator $f_{\psi}^{rel} : \mathcal{O} \times \mathcal{O} \times \mathcal{G} \rightarrow [-1, 1]$ compares consecutive frames (x_t, x_{t+1}) in the context of goal g to infer local progress or regression. The RRM is adaptive: it provides dense, non-zero feedback throughout the trajectory, detecting each small step toward the goal.

Technical Formulation

The combined reward at step t :

$$r_t = \alpha \cdot \sigma(f_\phi^{abs}(\text{enc}(x_t), \text{enc}(g))) + (1 - \alpha) \cdot \tanh(f_\psi^{rel}(\text{enc}(x_t), \text{enc}(x_{t+1}), \text{enc}(g)))$$

where $\text{enc}(\cdot)$ denotes a frozen CLIP encoder. The ARM is trained by minimizing squared error against VLM scalar pseudo-labels:

$$\mathcal{L}_{abs}(\phi) = \mathbb{E}_{(x,g,\hat{r})} \left[(f_\phi^{abs}(\text{enc}(x), \text{enc}(g)) - \hat{r})^2 \right]$$

The RRM is trained via binary cross-entropy against VLM pairwise preference labels $y \in \{0, 1\}$:

$$\mathcal{L}_{rel}(\psi) = -\mathbb{E}_{(x_t, x_{t+1}, g, y)} [y \log \sigma(f_\psi^{rel}) + (1 - y) \log(1 - \sigma(f_\psi^{rel}))]$$

Both networks are 2-layer MLPs applied to frozen CLIP features. The mixing coefficient $\alpha = 0.4$ is selected by validation.

Learning or Inference Procedure

1. Collect a replay buffer of (x_t, x_{t+1}, g) triples from random-policy rollouts in the target environment.
2. Query VLM (LLaMA-Vision-3.1-8B) offline to generate: (a) absolute pseudo-labels $\hat{r} \in [0, 1]$ for each frame; (b) pairwise preference labels $y \in \{0, 1\}$ for consecutive pairs.
3. Train f_ϕ^{abs} and f_ψ^{rel} on the VLM labels.
4. Run PPO with combined reward $r_t = \alpha f_\phi^{abs} + (1 - \alpha) f_\psi^{rel}$, querying only the lightweight reward MLPs (no VLM calls during RL).
5. Optionally fine-tune reward models every 10^5 steps using additional VLM labels from the current policy’s rollouts.

What the Guarantee Says

VLM-AR3L does not provide formal convergence guarantees, as these would require assumptions about the VLM’s labeling quality. The paper proves empirically that: (1) VLM pseudo-labels for both ARM and RRM are significantly more accurate than random ($>65\%$ agreement with human labels on held-out frames) and (2) the combined reward is more dense (non-zero at $>80\%$ of trajectory steps) and better correlated with episode return (Pearson $\rho = 0.71$) than either signal alone ($\rho = 0.52$ for ARM, $\rho = 0.48$ for RRM).

Experimental Findings

Task	VLM-AR3L	EUREKA	VLM-RM	Random
Combat Spider	51.4%	37.2%	41.8%	8.3%
Milk Cow	62.3%	48.1%	50.6%	12.1%
Shear Sheep	57.8%	41.0%	44.3%	9.7%
Hunt Cow	48.9%	36.8%	39.2%	7.5%

Task success rates averaged over 100 evaluation episodes. VLM-AR3L achieves 29–41% relative improvement over the next best VLM baseline (VLM-RM) across all four tasks.

Ablations and Interpretation

ARM only: Removing the RRM and using only absolute rewards reduces success rates by 22–28% (average 24%). Training is slower to converge due to sparse reward signal early in episodes.

RRM only: Removing the ARM and using only relative rewards reduces success rates by 18–32% (average 25%). Agents converge to oscillatory policies that score well on pairwise comparisons without completing tasks.

Mixing coefficient α : Performance peaks at $\alpha = 0.4$ (40% absolute, 60% relative). Lower α increases reward density but loses task-completion grounding; higher α restores grounding but reintroduces sparsity.

VLM size: Replacing LLaMA-Vision-3.1-8B with a 3B model reduces labeling quality (agreement with humans drops from 67% to 58%) and downstream success rates by approximately 8%, confirming that larger VLMs produce better reward supervision.

Reference

Kuan-Chen Chen, Winston Chen, Wei-Fang Sun, Min-Chun Hu. **VLM-AR3L: Vision-Language Models for Absolute and Relative Rewards in Reinforcement Learning**. arXiv:2607.00483, July 2026. IJCAI 2026. <https://arxiv.org/abs/2607.00483>

Keywords: model compression • binary quantization • large language models • spherical coding

BiSCo-LLM

Lookup-Free Binary Spherical Coding Brings Sub-1-Bit LLM Compression Within Reach

By Yuantian Shao, Peisong Wang, Zhilei Liu, Chuangyi Li, Yuanteng Chen, Pengcheng Xie, Yiwu Yao, Zhihui Wei & Jian Cheng (Chinese Academy of Sciences; University of CAS; Huawei Technologies)

arXiv:2607.08643 • arXiv preprint, July 2026

Large language models are memory-bound: frontier models require tens of gigabytes of weight storage, creating an insurmountable barrier for on-device and edge deployment. Existing extreme-quantization methods face a fundamental trade-off—codebook-based approaches achieve good quality but require storing a codebook and paying a lookup cost at inference; binary approaches eliminate lookup overhead but suffer severe quality degradation. BiSCo-LLM resolves this trade-off by introducing Binary Spherical Coding (BSCo), a lookup-table-free scheme that matches 2-bit codebook quality at 1-bit storage by representing weights as binary combinations of spherical frame vectors.

AT A GLANCE

Problem: Extreme LLM quantization forces a choice between codebook quality (with lookup overhead) or binary speed (with poor accuracy).

Why it matters: On-device deployment of large models is blocked by memory bandwidth; sub-1-bit compression at competitive quality would unlock a new class of edge applications.

Method: Binary spherical coding maps weight blocks onto a pre-defined tight frame of ± 1 vectors; decoded via XNOR-popcount, requiring no lookup table.

Result: LLaMA-3-8B at 1-bit storage within 1.6 perplexity points of 2-bit AQLM; 12× memory reduction versus FP16.

Evidence: Benchmarked on WikiText-2 perplexity and MMLU accuracy for LLaMA-3 (8B, 13B, 70B) and Mistral-7B.

The Big Picture

The deployment gap for large language models is primarily a memory problem. A 70-billion-parameter model in FP16 requires ~140 GB of GPU memory — prohibitive for all but the largest datacenter installations and completely out of reach for edge devices. Quantization compresses weight values to fewer bits, reducing memory proportionally.

At 4-bit (GPTQ, AWQ) and 2-bit (AQLM, QuIP) precision, existing methods achieve competitive accuracy, but even 2-bit precision requires ~17GB for a 70B model and a runtime codebook for reconstruction. True 1-bit methods (BitNet, OneBit) eliminate the codebook but degrade quality sharply because ± 1 scalar representations cannot capture the heterogeneous magnitude structure of LLM weights.

BiSCo-LLM breaks out of this binary by separating the *structure* of the code (binary on a sphere) from its *expressive power* (controlled by the number of frame vectors K).

Why Existing Approaches Fall Short

Codebook methods (AQLM, QuIP, VQ-LLM): Represent weight blocks as indices into a learnable codebook. Quality is excellent — codebooks can approximate diverse weight distributions — but the codebook itself must be stored in memory (typically 65,536 entries $\times$ 32 floats = 2 MB per layer) and queried at every matrix-vector multiply via a memory-bandwidth-intensive lookup.

Binary methods (BitNet, OneBit): Represent weights as single ± 1 scalars (possibly scaled per row or column). Zero lookup overhead and excellent throughput, but accuracy degrades sharply because all weight columns of different norms are mapped to the same code alphabet. The fundamental mismatch is that ± 1 cannot represent graded magnitudes.

Vector quantization without lookup: No prior work achieves this combination. The key missing insight is that binary *combinations* of vectors, rather than individual binary scalars, can approximate rich high-dimensional distributions while still decoding via purely binary arithmetic.

Core Mechanism

A binary spherical code represents each weight column $w \in \mathbb{R}^d$ as a sparse binary combination of pre-defined frame vectors $\{b_k\}_{k=1}^K \subset \{+1, -1\}^d$ that form a tight frame:

$$\hat{w} = \frac{1}{\sqrt{K}} \sum_{k=1}^K z_k b_k, \quad z_k \in \{0, 1\}$$

The frame property $\sum_k b_k b_k^\top = (K/d)I_d$ guarantees that the reconstruction operator is an isometry in expectation, distributing reconstruction error uniformly across all weight directions regardless of the weight norm distribution.

At runtime, decoding $\hat{w}$ requires only XNOR-popcount operations between z and the frame matrix — no lookup table, no floating-point multiply at decode time.

Technical Formulation

The binary tight frame satisfies:

$$\min_{\{b_k\} \subset \{+1, -1\}^d} \left\| \sum_{k=1}^K b_k b_k^\top - \frac{K}{d} I_d \right\|_F^2$$

This Grassmannian binary packing problem is solved once offline via a greedy sequential algorithm. Given the fixed frame, per-weight compression minimizes reconstruction error:

$$z^* = \operatorname{argmin}_{z \in \{0,1\}^K} \left\| w - \frac{1}{\sqrt{K}} \sum_{k=1}^K z_k b_k \right\|^2$$

During training with straight-through estimators:

$$\frac{\partial \mathcal{L}}{\partial z_k} \approx \frac{\partial \mathcal{L}}{\partial \hat{w}} \cdot \frac{b_k}{\sqrt{K}} \cdot \mathbf{1}[|z_k - 0.5| < 0.5]$$

The effective bit-rate is $R = (\log_2 K)/d$ bits per weight, tunable from ≈ 0.5 to ≈ 1.5 bits by varying $K \in [2^{d/3}, 2^{2d}]$.

Learning or Inference Procedure

1. **Frame construction (once):** Build a binary tight frame $\{b_k\}_{k=1}^K$ offline via greedy orthogonal pursuit.
2. **Initialization:** For each weight block W , solve the binary coding problem via fast iterative bit-flip optimization.
3. **Fine-tuning:** Train the binary codes z (not the frame) using STE gradients with a small calibration dataset (~ 512 sequences).
4. **Inference:** Replace each weight block with its binary code. Matrix-vector products computed as $\hat{W}x \approx \frac{1}{\sqrt{K}} B_z x$ where $B_z x$ is computed via XNOR-popcount on binary hardware.

What the Guarantee Says

For a d -dimensional weight vector and a random binary tight frame with K vectors, the expected reconstruction error is:

$$\mathbb{E}[\|w - \hat{w}\|^2] = \|w\|^2 \cdot \left(1 - \frac{K}{d \cdot 2^K} \sum_{s=0}^K \binom{K}{s} \left| \frac{s}{K} - 0.5 \right| \right)$$

This bound is tight and shows that reconstruction error vanishes as $K \rightarrow \infty$ (more frame vectors = higher quality), independently of the weight norm distribution.

Experimental Findings

Method (LLaMA-3-8B)	Bits	PPL
FP16	16	6.14
GPTQ-4bit	4	6.58
AQLM-2bit	2	7.21
BitNet-1bit	1	9.87
BiSCo-LLM 1bit	1	7.73
BiSCo-LLM 0.5bit	0.5	9.14

WikiText-2 perplexity (lower is better). BiSCo-LLM at 1-bit matches AQLM’s 2-bit quality within 0.5 PPL at half the memory footprint, and outperforms BitNet by 2.1 PPL at the same storage budget.

Ablations and Interpretation

Frame size K : Quality improves monotonically with K up to $K \approx 2d$, after which reconstruction error saturates. The sweet spot for 1-bit effective compression is $K = d$: each weight column uses d binary bits to select d frame vectors, yielding exactly 1 bit/weight.

Attention vs. MLP layers: Attention weights are more compressible (quality gap vs. FP16 smaller by 15%) than MLP weights, consistent with prior quantization literature. BiSCo-LLM supports layer-specific K tuning.

XNOR-popcount throughput: On a Snapdragon X Elite mobile SoC, BiSCo-LLM achieves 3.4× higher token generation throughput versus INT4 GPTQ, demonstrating that the lookup-free design translates directly to real hardware efficiency.

Model scale: Perplexity gap vs. FP16 decreases at larger model sizes (70B model: 1.8 PPL gap vs. 1.6 PPL gap for 8B at 1-bit), suggesting BiSCo-LLM scales favorably.

Reference

Yuantian Shao, Peisong Wang, Zhilei Liu, Chuangyi Li, Yuanteng Chen, Pengcheng Xie, Yiwu Yao, Zhihui Wei, Jian Cheng. **BiSCo-LLM: Lookup-Free Binary Spherical Coding for Extreme Low-Bit Large Language Model Compression**. arXiv:2607.08643, July 2026. <https://arxiv.org/abs/2607.08643>

Keywords: variational inference • continuous control • adaptive computation • reinforcement learning

Latent Memory Palace

Autoregressive Variational Inference Brings Chain-of-Thought Deliberation to Physical Control

By Chuning Zhu, Eva Xu, Jose Barreiros, Krishnan Srinivasan, Paarth Shah & Abhishek Gupta (Paul G. Allen School of CSE, University of Washington)

arXiv:2607.08724 • arXiv preprint, July 2026

Language models reason flexibly by generating variable-length chains of thought before producing an answer. Continuous control policies cannot: they map observations directly to actions in a fixed-depth computation graph, with no mechanism to think longer on harder problems. Latent Memory Palace (LMP) closes this gap by formalizing deliberation as autoregressive variational inference over a structured latent state space, enabling physical agents to allocate more or fewer computation steps depending on task complexity—achieving 16–26% higher success rates than fixed-depth recurrent baselines on long-horizon manipulation tasks.

AT A GLANCE

Problem: Continuous control policies cannot adaptively allocate deliberation time; they use the same computation depth on easy and hard decisions.

Why it matters: Long-horizon manipulation and dexterous tasks require proportionally deeper deliberation than routine motions, yet existing methods offer no mechanism for this.

Method: Hierarchical variational autoencoder with autoregressive latent steps and a learned halting mechanism trained via ELBO maximization.

Result: 47% higher success rate on long-horizon manipulation versus best recurrent baseline (LSTM); 16–26% across all tasks.

Evidence: Five-task benchmark spanning stacking, tool use, pick-place, deformable manipulation, and multi-object coordination.

The Big Picture

The gap between language model reasoning and physical control is not merely architectural — it is computational. A language model generating a plan for a complex task can produce a chain of thought of any length: simple tasks get short chains, complex tasks get long ones. This adaptive computation is free in the discrete-token genera-

tion paradigm where each token takes roughly the same wall-clock time.

Continuous control policies lack this flexibility entirely. A recurrent policy (LSTM, GRU, S4) updates a fixed-size hidden state once per time step. A diffusion policy iterates a fixed number of denoising steps. Neither can elect to “think harder” about a particularly tricky configuration. The result is policies that perform well on routine subtasks but fail catastrophically when unexpected complexity arises.

Latent Memory Palace formalizes variable-depth deliberation in the continuous domain by casting it as inference in a hierarchical latent variable model where the number of latent computation steps is itself a learned variable.

Why Existing Approaches Fall Short

Recurrent policies (LSTM, GRU, S4) maintain a hidden state that accumulates information over time but applies the same fixed-depth computation at each action step. The hidden state is informative but the per-step computation is shallow.

Model-based RL (Dreamer, TDMPC) plans over fixed-depth horizon rollouts. The horizon must be pre-specified and cannot adapt to task difficulty without expensive online search.

Diffusion policies (Diffuser) perform iterative denoising in action space. This iteration serves trajectory optimization, not task-level deliberation — the number of denoising steps is fixed and does not correlate with decision complexity.

Universal Transformers adapt computation depth in sequence modeling but have not been extended to continuous RL or equipped with the variational training objective needed for learning under environment uncertainty.

Core Mechanism

LMP introduces a *latent deliberation sequence*: before producing each action a_t , the agent runs K_t autoregressive latent computation steps $z_t^{(1)}, \dots, z_t^{(K_t)}$. Each latent step refines the agent’s internal representation of the task state, analogous to one token of chain-of-thought reasoning. The final latent state $z_t^{(K_t)}$ is then decoded to an action.

The number of steps K_t is itself learned: a halt probability $p_h(z_t^{(k)}, o_t) \in [0, 1]$ is computed at each step, and the agent halts when a Bernoulli sample fires. Harder observations (estimated via an auxiliary complexity measure) produce lower halt probabilities, naturally inducing more deliberation.

Technical Formulation

The model defines a hierarchical generative process with autoregressive latent prior:

$$p_{\theta}(z_t^{(1:K)} | o_t) = \prod_{k=1}^K p_{\theta}(z_t^{(k)} | z_t^{(1:k-1)}, o_t)$$

and posterior using retrospective information h_t^{ret} (future observations and rewards, available during training):

$$q_{\phi}(z_t^{(1:K)} | o_t, h_t^{\text{ret}}) = \prod_{k=1}^K q_{\phi}(z_t^{(k)} | z_t^{(1:k-1)}, o_t, h_t^{\text{ret}})$$

Training maximizes the ELBO plus behavioral cloning on expert actions:

$$\mathcal{L} = \mathbb{E}_{q_{\phi}} \left[\sum_t \log \pi_{\psi}(a_t | z_t^{(K_t)}) - \beta \sum_t \text{KL}(q_{\phi} \| p_{\theta})_t - \lambda_c \sum_t K_t \right]$$

where the $\lambda_c \sum_t K_t$ term penalizes excess computation, incentivizing economy in deliberation length.

Learning or Inference Procedure

1. Collect offline demonstrations of target tasks.
2. Train prior p_{θ} , posterior q_{ϕ} , policy π_{ψ} , and halt predictor p_h jointly via ELBO on demonstration trajectories.
3. At inference, use the prior p_{θ} only (no access to future): sample $z_t^{(k)} \sim p_{\theta}(\cdot | z_t^{(1:k-1)}, o_t)$ and halt when $p_h(z_t^{(k)}, o_t) > u$ for $u \sim U[0, 1]$.
4. Decode the final latent state to action via $\pi_{\psi}(a_t | z_t^{(K_t)})$.
5. Fine-tune with online RL (SAC) to improve beyond demonstrations.

What the Guarantee Says

LMP does not provide formal sample-complexity bounds. The paper proves empirically that: (1) The average deliberation depth $\bar{K}$ correlates positively with task difficulty (measured by expert-labeled complexity scores, Pearson $\rho = 0.73$ across tasks). (2) Performance improves monotonically as the maximum allowed deliberation budget $K_{\max}$ increases, with diminishing returns above $K_{\max} = 8$ for the tested tasks. (3) The computation overhead of LMP versus a fixed-depth LSTM is at most $2\times$ wall-clock time, as deliberation steps are shallow and the learned halting mechanism allocates deep deliberation only to a minority of high-complexity decisions.

Experimental Findings

Task	LMP	LSTM	S4	Dreamer
Stacking (5 obj.)	71.3%	48.5%	52.1%	44.2%
Multi-step tool use	63.7%	41.2%	45.6%	38.9%
Long-horizon P&P	78.4%	56.8%	61.3%	52.7%
Deformable manip.	55.1%	32.4%	37.8%	28.6%

Success rates (%) averaged over 5 seeds and 200 evaluation episodes. LMP exceeds the next-best baseline (S4) by 16–26% across all tasks, with the largest gains on deformable manipulation where unexpected contact events require the most reactive deliberation.

Ablations and Interpretation

Fixed vs. adaptive K : Replacing the learned halting mechanism with fixed $K = 4$ (the mean deliberation depth under adaptive halting) reduces performance by 11% on average, confirming that the *alignment* of depth with difficulty, not just depth on average, drives the gains.

ELBO vs. BC-only: Removing the ELBO training signal and using behavioral cloning alone (no latent structure) reduces success rates to near-LSTM levels, confirming the structured latent representation is essential.

Computation cost: The deliberation overhead is $1.4\text{--}1.9\times$ LSTM wall-clock time (task-dependent). The halting mechanism allocates $K \geq 6$ steps to only 12% of decisions, keeping average cost low.

Transferability: LMP policies pre-trained on one task set transfer to related tasks with 40% less fine-tuning data than LSTM baselines, suggesting the latent memory structure captures more transferable task representations.

Reference

Chuning Zhu, Eva Xu, Jose Barreiros, Krishnan Srinivasan, Paarth Shah, Abhishek Gupta. **Latent Memory Palace: Reasoning for Control as Autoregressive Variational Inference.** arXiv:2607.08724, July 2026. <https://arxiv.org/abs/2607.08724>